

JavaScript Map, Filter, and Reduce - Interview Guide

◆ Basic Concept Questions

1. What is `map()` in JavaScript?

Key Points:

- *Creates a new array*
- *Transforms each element*
- *Same length as original*
- *Returns modified values*
- *Non-mutating method*

The `map()` method is used to create a new array by applying a function to each element of the original array. It transforms every element according to the logic you provide and returns a brand new array with the same number of elements. Whatever you return from the callback function becomes the new value in the resulting array. It's really helpful when you want to modify or transform data without changing the original array.

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map(num => num * 2);
console.log(doubled); // [2, 4, 6, 8]
```

2. What is `filter()` in JavaScript?

Key Points:

- *Creates a new array*
- *Selects elements conditionally*
- *Returns only matching items*
- *Length can be smaller*
- *Uses boolean conditions*

The `filter()` method creates a new array containing only the elements that pass a certain condition or test. It checks each element against the condition you provide, and if the condition returns true, that element is included in the new array. The resulting array can be smaller than the original because it only keeps elements that match your criteria. This method is perfect when you want to extract specific items from an array based on some rules.

```
const numbers = [1, 2, 3, 4, 5, 6];
const evenNumbers = numbers.filter(num => num % 2 === 0);
console.log(evenNumbers); // [2, 4, 6]
```

3. What is `reduce()` in JavaScript?

Key Points:

- Reduces array to single value
- Uses accumulator pattern
- Processes left to right
- Needs initial value (optional)
- Most flexible method

The `reduce()` method processes an array and reduces it down to a single value by applying a function repeatedly. It maintains an accumulator that carries the result from one iteration to the next, combining all array elements into one final output. You can use it to calculate sums, build objects, flatten arrays, or perform any operation that needs to accumulate results. It's the most powerful among these three methods because it can do almost anything with array data.

```
const numbers = [1, 2, 3, 4];
const sum = numbers.reduce((accumulator, current) => accumulator + current, 0);
console.log(sum); // 10
```

4. Why are `map`, `filter`, and `reduce` called functional methods?

Key Points:

- Follow functional programming
- Don't mutate original data
- Use pure functions
- Promote declarative code
- Enable method chaining

These methods are called functional methods because they follow the principles of functional programming. They don't modify the original array but instead create new values or arrays, which makes them predictable and safe to use. They encourage you to write declarative code where you describe what you want to achieve rather than how to do it step by step. These methods accept functions as arguments and promote a cleaner, more readable coding style that's easier to test and debug.

```
const numbers = [1, 2, 3, 4, 5];
const result = numbers
  .filter(num => num > 2)
  .map(num => num * 2)
  .reduce((sum, num) => sum + num, 0);
console.log(result); // 24
```

5. Are `map`, `filter`, and `reduce` mutable or immutable?

Key Points:

- *Immutable methods*
- *Don't change original array*
- *Return new values/arrays*
- *Safe for data integrity*
- *Functional programming style*

All three methods are immutable, which means they never modify the original array. When you call any of these methods, the original array stays exactly the same, and you get a completely new array or value as the result. This immutability is a core principle that makes your code more predictable and prevents unexpected side effects. However, you need to be careful because if your array contains objects, the method doesn't create deep copies, so modifying nested objects can still affect the original data.

```
const original = [1, 2, 3];
const doubled = original.map(num => num * 2);
console.log(original); // [1, 2, 3] - unchanged
console.log(doubled); // [2, 4, 6] - new array
```

◆ Comparison Questions

1. What is the difference between `map()` and `forEach()`?

Key Points:

- *map returns new array*
- *forEach returns undefined*
- *map for transformation*
- *forEach for side effects*
- *map is chainable*

The main difference is that `map()` returns a new array with transformed values while `forEach()` returns undefined and is only used for side effects. When you use `map()`, you're creating something new based on the original array, but with `forEach()`, you're just performing actions like logging or updating external variables. You can chain other array methods after `map()` because it returns an array, but you

cannot chain after `forEach()`. Use `map()` when you need the transformed data, and use `forEach()` when you just want to do something with each element without creating a new array.

```
const numbers = [1, 2, 3];

const mapped = numbers.map(num => num * 2); // [2, 4, 6]
const looped = numbers.forEach(num => num * 2); // undefined
```

2. What is the difference between `map()` and `filter()`?

Key Points:

- *map transforms all elements*
- *filter selects elements*
- *map keeps same length*
- *filter changes length*
- *Different purposes*

The `map()` method transforms every single element in the array and always returns an array of the same length, while `filter()` selects only certain elements based on a condition and can return a shorter array. With `map()`, you're changing the values but keeping all elements, whereas with `filter()`, you're keeping the values but potentially removing elements. Think of `map()` as a transformation tool and `filter()` as a selection tool. You use `map()` when you want to modify data and `filter()` when you want to extract specific items.

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(num => num * 2); // [2, 4, 6, 8]
const evens = numbers.filter(num => num % 2 === 0); // [2, 4]
```

3. What is the difference between `filter()` and `reduce()`?

Key Points:

- *filter returns array*
- *reduce returns any value*
- *filter for selection*
- *reduce for accumulation*
- *reduce more flexible*

The `filter()` method always returns an array containing selected elements, while `reduce()` can return any single value like a number, string, object, or even an array. Filter is specifically designed for selecting elements based on conditions, but reduce is much more flexible and can perform filtering, mapping, summing, or any complex operation. While you can achieve filtering using `reduce()`, it's more

complex and less readable compared to using `filter()`. Use `filter()` when you just need to select items, and use `reduce()` when you need to accumulate or transform data into a single result.

```
const numbers = [1, 2, 3, 4, 5];

const evens = numbers.filter(num => num % 2 === 0); // [2, 4]
const sum = numbers.reduce((total, num) => total + num, 0); // 15
```

4. When would you prefer `reduce()` over `map()`?

Key Points:

- Need single value output
- Calculating totals/sums
- Building objects
- Flattening arrays
- Complex transformations

You should prefer `reduce()` over `map()` when you need to convert an array into a single value rather than another array of the same length. Common scenarios include calculating sums, averages, or counts, building objects from array data, or flattening nested arrays. While `map()` is great for one-to-one transformations, `reduce()` excels when you're aggregating or combining data. If your goal is to end up with something other than an array of transformed elements, reduce is usually the better choice.

```
const products = [
  { name: 'Apple', price: 50 },
  { name: 'Banana', price: 30 }
];

// reduce for total price
const total = products.reduce((sum, item) => sum + item.price, 0); // 80

// map would give array
const prices = products.map(item => item.price); // [50, 30]
```

5. Can `reduce()` replace both `map()` and `filter()`?

Key Points:

- Yes, technically possible
- Less readable code
- Not recommended
- map/filter more specific
- reduce for complex cases

Yes, technically `reduce()` can replace both `map()` and `filter()` because it's the most flexible method, but it's not recommended for simple use cases. When you use `reduce()` to replicate `map()` or `filter()`, the code becomes harder to read and understand compared to using the specific methods. The purpose of having `map()` and `filter()` is to make your intentions clear and your code more maintainable. You should only use `reduce()` when you actually need its power for complex operations or when you need to combine multiple operations efficiently in a single pass.

```
const numbers = [1, 2, 3, 4];

// Using reduce as map
const doubled = numbers.reduce((acc, num) => [...acc, num * 2], []); // [2, 4, 6, 8]

// Using reduce as filter
const evens = numbers.reduce((acc, num) => num % 2 === 0 ? [...acc, num] : acc, []); // [2, 4]

// Better to use map and filter directly
const doubledBetter = numbers.map(num => num * 2);
const evensBetter = numbers.filter(num => num % 2 === 0);
```

◆ Usage & Decision Questions

1. When should you use `map()`?

Key Points:

- Transform every element
- Need same array length
- Converting data types
- Extracting properties
- One-to-one mapping

You should use `map()` whenever you need to transform every element in an array and get a new array of the same length. Common scenarios include converting data types, extracting specific properties from objects, performing calculations on numbers, or formatting strings. It's perfect when you have an array and want to apply the same operation to each item without filtering anything out. Think of situations like doubling all numbers, extracting user names from user objects, or converting prices to different currencies.

```
const users = [
  { id: 1, name: 'John', age: 25 },
  { id: 2, name: 'Sarah', age: 30 }
];

const names = users.map(user => user.name); // ['John', 'Sarah']
```

2. When should you use `filter()`?

Key Points:

- *Select based on condition*
- *Remove unwanted items*
- *Find matching elements*
- *Validate array items*
- *Subset creation*

You should use `filter()` when you need to select only certain elements from an array based on specific criteria. It's ideal for removing unwanted items, finding all elements that match a condition, or creating subsets of data. Common use cases include filtering active users, selecting products within a price range, removing null values, or finding all even numbers. Anytime you're asking "which elements meet this condition", filter is your go-to method.

```
const products = [  
  { name: 'Laptop', price: 1000, inStock: true },  
  { name: 'Phone', price: 500, inStock: false },  
  { name: 'Tablet', price: 300, inStock: true }  
];  
  
const available = products.filter(product => product.inStock);  
// [{ name: 'Laptop', ... }, { name: 'Tablet', ... }]
```

3. When should you use `reduce()`?

Key Points:

- *Calculate totals/sums*
- *Convert to single value*
- *Build objects/maps*
- *Flatten nested arrays*
- *Complex aggregations*

You should use `reduce()` when you need to combine all array elements into a single value or when performing complex aggregations. It's perfect for calculating sums, averages, or counts, grouping data into objects, flattening nested arrays, or finding maximum or minimum values. Use reduce when your operation involves accumulating results or when you need more control than what `map()` or `filter()` provides. It's also great when you need to perform multiple operations in a single pass through the array.

```
const orders = [
  { item: 'Book', quantity: 2, price: 10 },
  { item: 'Pen', quantity: 5, price: 2 }
];

const totalCost = orders.reduce((total, order) => {
  return total + (order.quantity * order.price);
}, 0); // 30
```

4. Which method would you use to transform an array?

Key Points:

- Use `map()` method
- Transforms all elements
- Returns new array
- One-to-one conversion
- Preserves array length

You should use `map()` to transform an array because it's specifically designed for this purpose. Map applies a transformation function to every element and returns a new array with all the transformed values. Whether you're changing data types, performing calculations, or restructuring objects, map is the most clear and efficient choice. It makes your intention obvious to anyone reading the code and follows the single responsibility principle by focusing solely on transformation.

```
const celsius = [0, 10, 20, 30];
const fahrenheit = celsius.map(temp => (temp * 9/5) + 32);
console.log(fahrenheit); // [32, 50, 68, 86]
```

5. Which method would you use to remove elements from an array?

Key Points:

- Use `filter()` method
- Removes by condition
- Returns filtered array
- Keeps matching items
- Original stays intact

You should use `filter()` to remove elements from an array because it's designed to select elements based on conditions. You write a condition that returns true for elements you want to keep and false for elements you want to remove. The result is a new array with only the elements that passed your test, and the original array remains unchanged. This approach is clean, readable, and follows functional programming principles for safe data manipulation.


```
const numbers = [1, 2, 3, 4, 5, 6];
const withoutEvens = numbers.filter(num => num % 2 !== 0);
console.log(withoutEvens); // [1, 3, 5]
```

6. Which method would you use to calculate a sum or total?

Key Points:

- Use `reduce()` method
- Accumulates values
- Returns single number
- Perfect for aggregation
- Can handle complex logic

You should use `reduce()` to calculate sums or totals because it's built for accumulating values into a single result. It maintains a running total as it processes each element, which is exactly what you need for summation. You provide an initial value (usually 0) and add each element to the accumulator, and `reduce` handles the iteration and accumulation for you. This method is not only great for simple sums but also for calculating averages, products, or any other aggregate value.

```
const prices = [10, 20, 30, 40];
const total = prices.reduce((sum, price) => sum + price, 0);
console.log(total); // 100

// With objects
const cart = [
  { item: 'Apple', price: 5, qty: 2 },
  { item: 'Banana', price: 3, qty: 3 }
];
const cartTotal = cart.reduce((sum, item) => sum + (item.price * item.qty), 0);
console.log(cartTotal); // 19
```

◆ Behavior & Output Questions

1. What happens if `map()` callback does not return anything?

Key Points:

- Returns array of undefined
- Same length maintained
- No error thrown
- Not useful result
- Should always return value

If the `map()` callback doesn't return anything, JavaScript treats it as returning undefined, and you'll get an array filled with undefined values of the same length as the original array. This is usually a mistake because map is meant to transform values, not create undefined elements. The method won't throw an error, but the result won't be useful. This is why it's important to always explicitly return a value from your map callback to get meaningful transformed data.

```
const numbers = [1, 2, 3];
const result = numbers.map(num => {
  num * 2; // forgot to write return
});
console.log(result); // [undefined, undefined, undefined]

// Correct way
const correct = numbers.map(num => num * 2);
console.log(correct); // [2, 4, 6]
```

2. What happens if `filter()` callback returns a non-boolean value?

Key Points:

- *Converts to boolean*
- *Uses truthy/falsy rules*
- *Still works normally*
- *0, null, "" are falsy*
- *Non-zero numbers are truthy*

If the `filter()` callback returns a non-boolean value, JavaScript automatically converts it to a boolean using truthy and falsy rules. Values like 0, null, undefined, empty string, and NaN are considered falsy and will exclude the element, while all other values including non-zero numbers, non-empty strings, and objects are truthy and will include the element. This behavior can be useful but also confusing if you're not aware of it, so it's better to return explicit boolean values for clarity.

```
const numbers = [0, 1, 2, 3, 4, 5];

// Returns numbers (truthy/falsy evaluation)
const result = numbers.filter(num => num); // [1, 2, 3, 4, 5] (0 is falsy)

// Better to be explicit
const betterResult = numbers.filter(num => num > 0); // [1, 2, 3, 4, 5]
```

3. What happens if `reduce()` is called without an initial value?

Key Points:

- *Uses first element as initial*

- Starts from second element
- Empty array throws error
- Can cause unexpected results
- Better to provide initial value

If you call `reduce()` without an initial value, it uses the first element of the array as the initial accumulator value and starts processing from the second element. This works fine for operations like sum, but it can cause problems with empty arrays because there's nothing to use as the initial value, and JavaScript will throw an error. It's generally safer and clearer to always provide an initial value to avoid edge cases and make your code more predictable.

```
const numbers = [1, 2, 3, 4];

// Without initial value
const sum1 = numbers.reduce((acc, num) => acc + num); // 10 (starts with 1)

// With initial value (recommended)
const sum2 = numbers.reduce((acc, num) => acc + num, 0); // 10

// Empty array without initial value - ERROR
const empty = [];
// const error = empty.reduce((acc, num) => acc + num); // TypeError

// Empty array with initial value - SAFE
const safe = empty.reduce((acc, num) => acc + num, 0); // 0
```

4. What will `reduce()` return if the array has only one element?

Key Points:

- Returns that element directly
- Callback never runs
- With initial value: runs once
- Without initial: just returns element
- No transformation applied

If the array has only one element and you don't provide an initial value, `reduce()` simply returns that element without calling the callback function at all. If you do provide an initial value, the callback runs once with the initial value as the accumulator and the single element as the current value. This behavior is important to understand because it means your callback might not execute at all in some cases, which could lead to unexpected results if you're not careful.

```
const single = [5];

// Without initial value - returns the element directly
const result1 = single.reduce((acc, num) => acc + num);
console.log(result1); // 5 (callback never runs)

// With initial value - callback runs once
const result2 = single.reduce((acc, num) => acc + num, 10);
console.log(result2); // 15 (10 + 5)
```

5. Can `map()` or `filter()` change the original array?

Key Points:

- *Methods don't mutate original*
- *But can modify nested objects*
- *Shallow copy created*
- *Reference types affected*
- *Deep clone needed for safety*

No, `map()` and `filter()` do not change the original array itself, they always create and return a new array. However, if the array contains objects or arrays, the methods create a shallow copy, which means the objects inside are still references to the original objects. If you modify properties of these objects inside the callback, you will affect the original data. To completely avoid mutations, you need to create deep copies of objects when transforming them.

```
const users = [{ name: 'John', age: 25 }];

// map doesn't change array, but can modify objects
const updated = users.map(user => {
  user.age = 26; // This modifies original object!
  return user;
});

console.log(users[0].age); // 26 (original changed)

// Correct way - create new object
const correct = users.map(user => ({
  ...user,
  age: 27
}));

console.log(users[0].age); // 26 (original not changed)
console.log(correct[0].age); // 27 (new object)
```

◆ Callback & Parameters

1. What parameters are passed to the callback of `map()`?

Key Points:

- *currentValue (element)*
- *index (position)*
- *array (original array)*
- *Usually use first only*
- *All are optional to use*

The `map()` callback receives three parameters: the current element being processed, the index of that element, and the entire original array. The first parameter is the current value you're transforming, the second is its position in the array, and the third is a reference to the complete array. In most cases, you only need the current element, but the index is useful when you need to know the position, and the array parameter helps when your transformation depends on other elements or the array's overall state.

```
const numbers = [10, 20, 30];

const result = numbers.map((element, index, array) => {
  console.log(`Element: ${element}, Index: ${index}, Array: ${array}`);
  return element * index;
});

// Output:
// Element: 10, Index: 0, Array: 10,20,30
// Element: 20, Index: 1, Array: 10,20,30
// Element: 30, Index: 2, Array: 10,20,30

console.log(result); // [0, 20, 60]
```

2. What parameters are passed to the callback of `filter()`?

Key Points:

- *currentValue (element)*
- *index (position)*
- *array (original array)*
- *Returns boolean condition*
- *Index useful for comparisons*

The `filter()` callback also receives three parameters: the current element, its index, and the original array. These are the same parameters that `map()` receives, but with `filter()`, your callback should return a boolean value to determine whether to include the element. The current element is what you're testing, the index can help with position-based filtering, and the array reference allows you to compare the current element with other elements in the array for more complex filtering logic.

```
const numbers = [1, 2, 3, 4, 5];

const result = numbers.filter((element, index, array) => {
  // Filter elements greater than the element at index 2
  return element > array[2];
});

console.log(result); // [4, 5]

// Common use - just element
const evens = numbers.filter(num => num % 2 === 0); // [2, 4]
```

3. What parameters are passed to the callback of `reduce()`?

Key Points:

- *accumulator (running result)*
- *currentValue (element)*
- *index (position)*
- *array (original array)*
- *Accumulator most important*

The `reduce()` callback receives four parameters: the accumulator (the running result), the current element, the index, and the original array. The accumulator holds the value that's being built up as you process each element, and it becomes the final return value. The current element is what you're processing right now, while index and array serve the same purpose as in other methods. The accumulator is what makes reduce powerful because it carries state across iterations, allowing you to build up complex results.

```
const numbers = [1, 2, 3, 4];

const result = numbers.reduce((accumulator, currentValue, index, array) => {
  console.log(`Acc: ${accumulator}, Current: ${currentValue}, Index: ${index}`);
  return accumulator + currentValue;
}, 0);

// Output:
// Acc: 0, Current: 1, Index: 0
// Acc: 1, Current: 2, Index: 1
// Acc: 3, Current: 3, Index: 2
// Acc: 6, Current: 4, Index: 3

console.log(result); // 10
```

4. What is the role of the accumulator in `reduce()`?

Key Points:

- *Stores intermediate result*
- *Carries state across iterations*

- *Becomes final return value*
- *Updated each iteration*
- *Initialized by initial value*

The accumulator in `reduce()` is a variable that stores the intermediate result as the method processes each element of the array. It starts with either the initial value you provide or the first element of the array, and then it gets updated with whatever you return from your callback function after processing each element. The accumulator carries this running result through all iterations and whatever value it holds after processing the last element becomes the final return value. This is what allows reduce to build up complex results like sums, objects, or any accumulated data structure.

```
const numbers = [1, 2, 3, 4];

// Accumulator as sum
const sum = numbers.reduce((acc, num) => {
  return acc + num; // accumulator grows with each addition
}, 0); // starts at 0
console.log(sum); // 10

// Accumulator as object
const fruits = ['apple', 'banana', 'apple', 'orange', 'banana'];
const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1; // accumulator builds object
  return acc;
}, {}); // starts as empty object
console.log(count); // { apple: 2, banana: 2, orange: 1 }
```